Lagos Life Social Network — Product & Engineering Context

Purpose of this document

Use this document as the initial product brief and engineering context for a coding assistant (Cursor). The goal is to explore and, after validation, build an independent social platform for players of **Lagos Life**.

Important: This is a concept brief, not evidence that Lagos Life currently permits third-party social integrations. Treat all platform API access as unconfirmed until the Lagos Life team provides documentation or written approval.

1. Product concept

Build a standalone, mobile-first social network where people can represent and connect through their Lagos Life in-game characters. Think of it as a social layer for the game—not a generic Instagram clone.

Players could:

- Create an account and a social profile.
- Link their profile to an existing Lagos Life character, if the game supports an official verification method.
- Share screenshots and photos from their in-game life.
- Follow other players/characters.
- Like and comment on posts.
- Discover profiles and content.
- Report content, block users, and manage privacy.

The product should remain useful even if official API access is delayed: the first version can support ordinary accounts and self-reported character details, clearly labelled as **unverified**. Never imply a character is verified unless an official, reliable verification flow exists.

2. Problem and opportunity

Lagos Life is a game with a social, identity, and role-play dimension. Players may want a place to:

- Keep a persistent identity for their character outside the game.
- Share screenshots, outfits, achievements, and stories.
- Find other players and follow their characters.
- Continue conversations and community interactions outside a live game session.

The hypothesis is that a game-specific social identity and feed would be more compelling than a general-purpose social app because the profiles and content are centred on Lagos Life.

This is a hypothesis to validate—not an established fact. Before investing heavily, interview players and test whether they would actively post, follow, and return to the platform.

3. MVP scope

Include in the first usable version

1. **Authentication:** email/password or a trusted auth provider; secure sessions and account recovery.
2. **User profile:** handle, display name, bio, avatar, optional Lagos Life character name and screenshot.
3. **Verification status:** `unlinked`, `self_reported`, or `verified`. Only allow `verified` after an approved, server-side verification mechanism is available.
4. **Posts:** image upload, caption, timestamps, author profile.
5. **Social graph:** follow/unfollow users.
6. **Feed:** posts from followed users, with pagination.
7. **Interactions:** likes and comments.
8. **Discovery:** search profiles and browse recent/public posts.
9. **Safety basics:** report post/user, block user, delete own post, basic rate limits and moderation controls.
10. **Responsive UI:** prioritize mobile screens; the target audience is likely to use phones.

Explicitly out of scope for MVP

- Direct messages and live chat.
- Real-time game state, player location, relationships, jobs, inventory, or activity feeds.
- In-game currency, payments, trading, or game account credentials.
- Scraping or reverse-engineering private game APIs.
- Automated bot accounts or posting on behalf of players.
- Any claim that the app is official or endorsed by Lagos Life.

These can be considered later only if demand, safety, permissions, and technical access justify them.

4. Lagos Life integration: key unknowns

The integration is the largest dependency. Do not build the core product around assumptions about APIs.

Questions for the Lagos Life team:

1. Is there an official third-party developer API or SDK, and is it open to independent developers?
2. Can an external app authenticate a player or verify ownership of a Lagos Life account/character through OAuth, signed tokens, a deep link, or another approved flow?
3. Which character fields, if any, can be read: stable character ID, display name, avatar, profile data, achievements, or other public attributes?
4. Are webhooks or event APIs available? If so, which events and what are the rate limits?
5. Are external social platforms permitted under the game's terms, branding rules, privacy rules, and developer policies?
6. Is a formal partnership, approval, or attribution required?
7. Are there test/sandbox accounts, API quotas, and production access requirements?

Integration rules

- Do not ask users for their Lagos Life password.
- Do not scrape endpoints or reverse-engineer private APIs.
- Do not expose API keys or secrets in frontend code.
- Verify any identity proof on the server.

- Store only the minimum third-party data needed; define retention and deletion behaviour.
- Make verification revocable if the official platform supports it.
- Use clear labels for unverified versus verified identities.
- Keep a clean integration boundary so the app works without Lagos Life API access.

Suggested integration abstraction

Design an interface such as `GameIdentityProvider` with methods conceptually equivalent to:

- `beginLinking(userId)`
- `completeLinking(callbackData)`
- `getLinkedIdentity(userId)`
- `unlinkIdentity(userId)`

Implement a mock/local provider first. Add a real provider only after official API documentation and permission are confirmed. Do not invent endpoint URLs or request/response schemas.

5. Suggested technical direction

These are starting recommendations, not hard constraints. Inspect the existing repository and follow its established conventions before introducing frameworks or replacing architecture.

Backend

- TypeScript with Node.js.
- PostgreSQL as the primary relational database.
- Object storage for uploaded images (use a provider abstraction rather than coupling business logic to one vendor).
- REST API for the initial product; add realtime functionality only when there is a demonstrated need.
- Background jobs for image processing, moderation workflows, and cleanup if required.
- Database migrations checked into source control.
- Structured logs, request IDs, validation, and centralized error handling.

Frontend

- Responsive web app, mobile-first.
- Choose the framework based on the repository; if starting from scratch, use a mature TypeScript framework.
- Use accessible forms, loading states, empty states, and clear error feedback.
- Do not place secrets in client-side environment variables.

Engineering principles

- Prefer simple, explicit SQL and clear data access patterns.
- Enforce important invariants in the database as well as application code.
- Use pagination for feeds and search results.
- Use idempotent handling for uploads and external callbacks.
- Add tests for permissions, ownership, verification status, and feed visibility.
- Keep abstractions purposeful; avoid unnecessary helper layers.

- Never log passwords, session tokens, signed URLs, or private API credentials.

6. Initial data model

A starting point for discussion; adjust to the actual codebase and product decisions.

- `users`: `id`, `email`, `password_hash` or auth-provider identifier, `created_at`, `status`
- `profiles`: `user_id`, `handle`, `display_name`, `bio`, `avatar_url`, `created_at`, `updated_at`
- `game_identities`: `id`, `user_id`, `provider`, `external_character_id` (nullable until verified), `character_name`, `verification_status`, `verified_at`, `metadata` (only approved, non-sensitive fields), timestamps
- `posts`: `id`, `author_id`, `caption`, `visibility`, `created_at`, `updated_at`, `deleted_at`
- `post_media`: `id`, `post_id`, `storage_key`, `media_type`, `width`, `height`, `created_at`
- `follows`: `follower_id`, `followed_id`, `created_at`; unique constraint on the pair; prevent self-follow
- `likes`: `user_id`, `post_id`, `created_at`; unique constraint on the pair
- `comments`: `id`, `post_id`, `author_id`, `body`, `created_at`, `updated_at`, `deleted_at`
- `blocks`: `blocker_id`, `blocked_id`, `created_at`; unique constraint on the pair
- `reports`: `id`, `reporter_id`, `target_type`, `target_id`, `reason`, `details`, `status`, `created_at`

Important data-model considerations:

- Use immutable internal user IDs; handles can change.
- Do not use a character name as a unique identity unless the official game guarantees uniqueness.
- Use an official stable external ID only if the platform provides one and allows storage.
- Define feed visibility and block behaviour before implementing queries.
- Decide whether posts are public or follower-only; default to a safe, understandable option.
- Add indexes based on real queries, such as feed ordering, handle lookup, and relationship lookups.

7. Core user journeys

Journey A: Join and create a profile

1. User signs up and verifies their email, if applicable.
2. User chooses a unique handle and display name.
3. User optionally adds a character name and image.
4. The UI clearly marks the game identity as unverified until official verification is completed.

Journey B: Share a post

1. User selects an image and caption.
2. Server validates file type, size, and permissions.
3. Image is stored in object storage; server stores metadata.
4. Post appears in the user's profile and eligible feeds.
5. User can delete their own post.

Journey C: Follow and browse

1. User searches a handle or browses discovery.
2. User follows a profile.
3. Feed shows eligible posts from followed accounts in reverse chronological order.
4. Blocking or privacy settings override feed visibility.

Journey D: Link a Lagos Life character

1. User starts linking from settings.
2. If no official integration exists, show “Character linking is not available yet” or allow self-reported details marked unverified.
3. If an official flow becomes available, complete the flow server-side and verify the result before changing the status to `verified`.

8. Security, privacy, and moderation

Treat user-generated content and account security as core features, not later polish.

- Validate and limit uploaded file types and sizes; do not trust file extensions alone.
- Consider image metadata removal and image processing.
- Apply rate limits to authentication, posting, comments, follows, likes, and reports.
- Use authorization checks on every mutation and private-resource read.
- Prevent IDOR vulnerabilities: a user must not be able to edit/delete another user's resources by changing an ID.
- Use safe output encoding and avoid rendering untrusted HTML.
- Provide report, block, and account-deletion flows.
- Establish content rules and an admin moderation path before public launch.
- Minimize personal data; do not collect precise real-world location or sensitive game data unless necessary and explicitly justified.
- Provide a privacy policy and terms before public release.
- Plan backups, restore testing, and operational monitoring.

9. Product validation before scaling

Test the main assumption before building advanced integration:

- Speak to 10–20 active Lagos Life players.
- Show a lightweight mockup of profile, feed, and post creation.
- Ask whether they would post at least weekly, whom they would follow, and what would make them return.
- Recruit a small group of testers and measure activation, first-post rate, follows per user, weekly retention, and reports/blocks.
- Validate the game's terms and API permissions before marketing the product as connected to Lagos Life.

10. Proposed implementation phases

Phase 0 — Discovery

- Inspect the repository.
- Confirm the chosen frontend/backend stack and deployment target.
- Document API access as a dependency and keep it mocked.
- Validate the core social use case with players.

Phase 1 — Foundation

- Authentication, profiles, database migrations, API validation, error handling, and tests.

Phase 2 — Social MVP

- Image posts, follow graph, chronological feed, likes, comments, profile discovery.

Phase 3 — Trust and launch readiness

- Reports, blocks, moderation tools, rate limits, privacy controls, observability, backups, and deployment checklist.

Phase 4 — Official integration (only after approval)

- Implement the real identity provider using official docs.
- Add identity verification, approved public data, and supported event integrations only.
- Test revoked access, API failure, rate limits, and account deletion.

11. Instructions for Cursor

Start by inspecting the existing repository and report:

1. The current stack and folder structure.
2. What is already implemented.
3. Missing pieces relative to the MVP.
4. Security or data-model risks.
5. A small, prioritized implementation plan.

Then proceed incrementally. Do not rewrite the whole project without a clear reason. Before coding:

- Ask for clarification only when a product decision blocks progress.
- State assumptions explicitly.
- Do not invent Lagos Life API endpoints, credentials, schemas, or permissions.
- Build a mock `GameIdentityProvider` until official integration details are confirmed.
- Prefer a working, tested vertical slice over broad scaffolding.
- Add tests for each important feature and authorization rule.
- Keep this document updated as product decisions change.

First task for Cursor

“Inspect this repository and propose a concrete, phased plan for the Lagos Life social network MVP described in this document. Do not implement anything yet. Identify existing code we can reuse, the key design decisions, and any unknowns that could block development.”